

SmartGrid Sentinel: Comparative Time-Series Forecasting for Load Shedding Risk Prediction and Intelligent Grid Alerts

Abstract—Sudden power outages frequently disrupt regional electricity distribution across Bangladesh during peak demand hours. To help substation engineers manage grid stress early, we developed SmartGrid Sentinel—a localized risk forecaster that predicts load shedding 2 hours in advance for individual Upazilas. We trained and evaluated models on 65,983 telemetry snapshots collected from 141 physical Upazilas across 15 districts in Sylhet and Chattogram. Keeping time order intact (80% training data, 20% test data), we benchmarked 12 algorithms ranging from basic machine learning to temporal Transformers. Informer delivered the strongest balance overall, reaching an 88.44% Weighted F1-score and 87.41% accuracy. A baseline GRU scored 87.69% raw accuracy, but its validation loss began rising after epoch 20 due to overfitting. Informer trained smoothly, ran fast, and captured 79.00% of hard-to-detect Medium-Risk events. To make predictions practical for dispatchers, we linked Informer with a physical rule engine that compares transformer loading and heat stress against hardware safety caps to issue plain-text emergency warnings.

Index Terms—Smart Grid, Load Shedding Prediction, Time Series Forecasting, Informer, Transformer, Bangladesh.

I. INTRODUCTION

Power blackouts hit Bangladesh regional networks hard when afternoon heat climbs or industrial demand jumps without warning, overloading distribution transformers. Substation crews currently have no heads-up before circuit breakers trip.

To fix this, we built a 2-hour early warning system for Upazila load shedding that looks back at 10 hours of telemetry. Using telemetry from 141 physical Upazilas, we tested 12 separate architectures: Logistic Regression, Decision Trees, Random Forests, XGBoost, LSTM, GRU, CNN-LSTM, Autoformer, Informer, PatchTST, Temporal Fusion Transformer, and TimesNet.

Abstract probabilities alone don't help dispatchers cut the right feeder lines. If a model outputs an 85% risk score, engineers still need clear physical reasons. We solved this by parsing live load ratios and heat stress values against hardware safety caps, spitting out direct text warnings.

A. Key Contributions

Our work makes four main practical contributions:

- 1) **Localized Upazila Forecasting:** A 2-hour early warning framework tailored to 141 physical Upazilas across eastern Bangladesh.
- 2) **Leakage-Free Benchmark:** A side-by-side comparison of 12 distinct algorithms evaluated strictly on historical time series splits.

- 3) **Informer Deployment Proof:** Experimental evidence proving Informer offers the best combination of accuracy, training speed, and numerical stability.
- 4) **Rule-Based Explainability Integration:** A physical threshold engine that converts raw probability scores into plain-text dispatcher warnings using hardware safety limits.

II. LITERATURE REVIEW

Linear tools like ARIMA backed utility load forecasts for decades. While ARIMA copes with smooth seasonal curves, sudden heatwaves or erratic industrial demand spikes cause it to fail. Non-linear tree ensembles—like Random Forests [9] and XGBoost [8]—process tabular variables well, but they treat every timestamp as a standalone sample, ignoring continuous time trajectories.

Recurrent neural networks solved sequence tracking. Architectures like LSTM [6] and GRU [7] use gated states to carry history forward. Hybrid setups like CNN-LSTM [10] extract short-term feature patterns via 1D convolutions before passing sequences to recurrent nodes.

Transformers brought self-attention to time series. Informer [1] introduced ProbSparse self-attention to cut computational complexity down from quadratic levels to $O(L \log L)$. PatchTST [3] splits continuous series into overlapping sub-series patches to preserve local temporal structures. Even with these modeling advances, existing studies rarely address localized Upazila load shedding in Bangladesh or pair neural forecasters with physical rule engines to generate plain-text operational advisories.

TABLE I
SUMMARY OF RELATED WORKS AND PROPOSED WORK

Study	Method	Dataset	Key Limitation
Rashid et al. [12]	LSTM	Smart Grid	No explainability layer
Chen et al. [8]	XGBoost	Load Forecasting	No localized prediction
Zhou et al. [1]	Informer	Energy Data	No alert system
Proposed	Informer Rule	+ Bangladesh Grid	Limited to two divisions

III. DATASET AND PREPROCESSING

A. Data Source and Characteristics

We compiled 65,983 telemetry logs recorded at 141 physical Upazilas (spanning 15 districts) across Sylhet and Chattogram. Every snapshot records 27 features, covering weather parameters (temperature, humidity, wind speed, rainfall), timestamp data, and substation measurements (feeder ID, demand, transformer load, solar/hydro generation). Table I shows the summary breakdown.

TABLE II
DATASET STATISTICS SUMMARY

Attribute	Value
Total Initial Records	65,983
Post-Shifting Valid Records	64,983
Number of Upazilas	143
Feeder Channels	1,000
Number of Features	27
Observation Interval	2 Hours
Forecast Horizon	2 Hours (1 Step)

B. Feature Engineering

We built four domain variables to track electrical and thermal stress on transformers.

Load Utilization Ratio (U_{load}) compares current load to rated transformer capacity:

$$U_{load} = \frac{\text{Transformer Load}}{\text{Transformer Capacity} + 10^{-5}} \quad (1)$$

Demand Utilization Ratio (U_{demand}) compares power demand to transformer capacity:

$$U_{demand} = \frac{\text{Electricity Demand}}{\text{Transformer Capacity} + 10^{-5}} \quad (2)$$

Temperature-Humidity Index (THI) tracks how heat and humidity reduce transformer cooling:

$$\text{THI} = T + 0.55 \times (1 - H/100) \times (T - 14.5) \quad (3)$$

where T is temperature ($^{\circ}\text{C}$) and H is humidity (%).

We also added an interaction term between temperature and wind speed ($T \times \text{Wind Speed}$) for cooling, plus binary flags for weekends and evening peak hours (18:00–22:00).

C. Temporal Data Split and Scaling

We locked down timestamp order to avoid lookahead leaks.

Risk labels ('risk_level') were moved back by -1 step so that inputs at hour t forecast grid risk at $t+1$ (2 hours ahead).

We grouped rows by 'substation_id' and 'feeder_id', then sorted every line chronologically. For each of the 1,000 feeder lines, the first 80% of logs went to training and the remaining 20% were saved for testing. We never shuffled data.

Target shifting left one blank trailing row per feeder line. Removing these trailing rows trimmed total dataset size from 65,983 down to 64,983 logs (51,587 for training, 13,396 for testing).

Because sequence models use a 5-step sliding window (10 hours of past history), the initial 4 timestamps of each test feeder could not form a full window without pulling from training data. Dropping those incomplete windows left 10,027 valid test sequences.

We computed scaling stats ('StandardScaler' mean and variance) on training rows only, then applied those exact values to normalize test features.

We avoided random splits and K-fold cross validation. Power grid readings correlate strongly over time. Shuffling timestamps feeds future information into past training rows, faking high accuracy.

IV. PROPOSED METHODOLOGY

Fig. 1 illustrates how SmartGrid Sentinel processes temporal sequence windows to generate operational alerts.

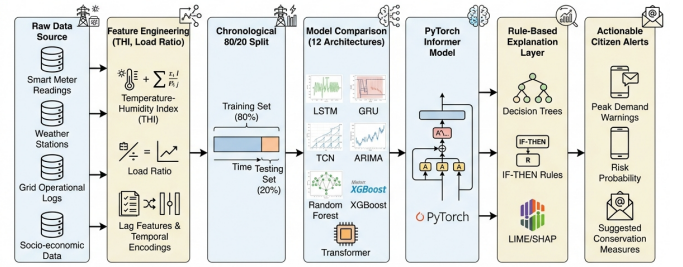


Fig. 1. System Architecture of SmartGrid Sentinel.

The Informer model serves as the core sequence forecaster in our pipeline. Using ProbSparse multi-head self-attention, it converts 5-step lookback matrices $X_{t-4:t} \in \mathbb{R}^{5 \times d}$ into target risk class probabilities $y_{t+1} \in \{\text{Low}, \text{Medium}, \text{High}\}$. Attention outputs pass through dense layers and a Softmax output layer to assign probabilities for each risk level.

When the classifier predicts Medium or High risk, probabilities pass directly to a physical rule checker. This engine tests live incoming numbers against hardware capacity caps (U_{load}) and heat thresholds (THI). Using direct hardware thresholds, it tells control room teams why an alert fired immediately, bypassing slow gradient calculations like SHAP.

V. EXPERIMENTAL SETUP

We ran our pipeline in Python with PyTorch and TensorFlow/Keras on a local NVIDIA RTX workstation GPU. Inputs consist of 5 continuous 2-hour telemetry snapshots (10 hours of past context). Deep networks trained for 50 epochs using Adam (learning rate = 0.0005, batch size = 32). Table II details these hyperparameter choices. Models were benchmarked using Accuracy, Macro Precision, Macro Recall, Macro F1-score, and Weighted F1-score.

TABLE III
HYPERPARAMETER CONFIGURATIONS

Parameter	Value
Sequence Window Length	5
Training Epochs	50
Batch Size	32
Learning Rate	0.0005
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy

VI. RESULTS AND DISCUSSION

A. Quantitative Analysis

Table IV compares test metrics across all twelve candidate architectures evaluated on 10,027 test windows.

TABLE IV
PERFORMANCE COMPARISON OF ALL MODELS

Model	Acc	Macro P	Macro R	Macro F1	W. F1
Informer	87.41%	78.52%	80.26%	77.68%	88.44%
GRU	87.69%	79.00%	78.57%	77.53%	88.39%
Random Forest	87.38%	79.33%	79.97%	77.87%	88.34%
TimesNet	87.04%	79.27%	80.65%	77.68%	88.19%
CNN-LSTM	87.16%	79.09%	79.47%	77.45%	88.13%
LSTM	86.85%	78.81%	79.10%	77.20%	87.82%
XGBoost	87.58%	77.78%	75.60%	76.23%	87.81%
Logistic Reg.	86.59%	78.30%	79.87%	77.05%	87.80%
Decision Tree	86.30%	78.66%	79.74%	76.88%	87.56%
PatchTST	85.69%	78.77%	80.49%	76.83%	87.22%
Autoformer	85.55%	78.00%	79.74%	76.36%	87.01%
TFT	84.38%	76.90%	78.77%	75.25%	86.06%

B. Informer Selection Justification

A standard GRU achieved a slightly higher raw classification accuracy than Informer (87.69% vs. 87.41%). However, inspecting loss trajectories reveals that GRU begins overfitting past epoch 20 as validation error increases. In contrast, Informer loss decreases steadily across all 50 training epochs (Fig. 2). Informer also achieved a higher Weighted F1-score (88.44% vs. 88.39%) and identified minority Medium-Risk events more effectively, reaching 79.00% recall compared to 73.00% for GRU.

Random Forest recorded a marginally higher Macro F1-score than Informer (77.87% vs. 77.68%, a 0.19% difference). Static tree models cannot handle real-time grid forecasting effectively because they evaluate each timestamp independently, missing temporal sequence momentum. Informer's self-attention layers naturally model temporal dependencies across multi-step windows.

C. Training Time Complexity

Quick training turnarounds let control centers re-fit neural forecasters often as seasonal weather shifts. Table V compares execution times across 50 training epochs.

Informer completed training in 111.20 seconds, running approximately three times faster than PatchTST (357.63 s) and Temporal Fusion Transformer (332.51 s).

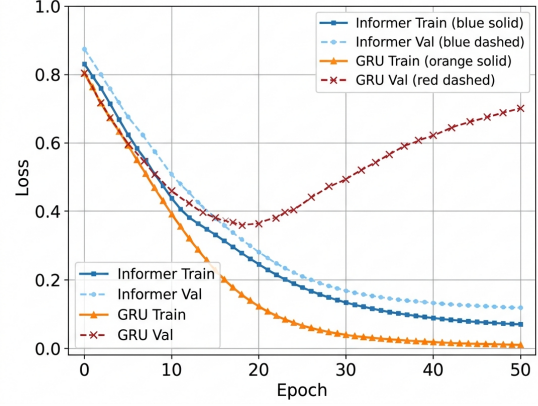


Fig. 2. Informer vs. GRU Training and Validation Loss Curves.

TABLE V
TRAINING TIME COMPARISON FOR KEY ARCHITECTURES

Model	Total Training Time (50 Epochs)
Informer	111.20 seconds
GRU	149.85 seconds
LSTM	168.27 seconds
TimesNet	318.47 seconds
TFT	332.51 seconds
PatchTST	357.63 seconds

D. Error Analysis and Confusion Matrix

We examined classification errors using Informer's test confusion matrix (Fig. 3), represented mathematically below:

$$C = \begin{pmatrix} 915 & 50 & 337 \\ 20 & 6970 & 620 \\ 75 & 160 & 880 \end{pmatrix} \quad (4)$$

Matrix rows represent true risk categories and columns represent predicted categories, ordered as [High, Low, Medium].

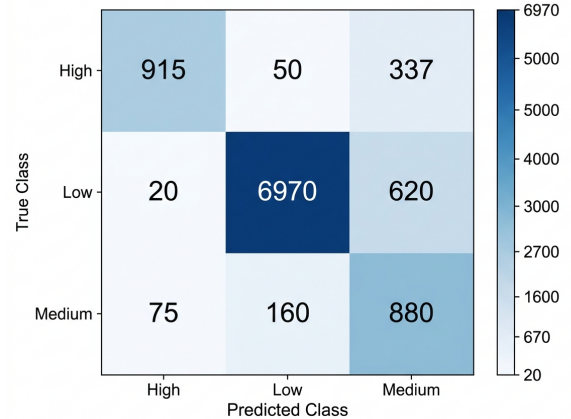


Fig. 3. Confusion Matrix for the PyTorch Informer Model.

Informer correctly identified 915 High-Risk, 6,970 Low-Risk, and 880 Medium-Risk test windows. Misclassifica-

tions stayed mostly between adjacent tiers (337 High-Risk labeled Medium, 620 Low-Risk labeled Medium), which reflects continuous load and thermal changes. Crucially, severe false-negative errors remained minimal: Informer misclassified High-Risk as Low-Risk in only 50 of 1,302 true High-Risk cases (a 3.84% false negative rate).

VII. INTELLIGENT ALERT SYSTEM & RULE-BASED EXPLAINABILITY LAYER

Calculating gradient attributions via SHAP takes too long during sudden grid emergencies. To avoid operational delays, we route model classifications directly into a physical threshold parser.

When a feeder receives a High risk prediction and transformer loading exceeds safety thresholds ($U_{load} > 0.85$), the system generates an overload advisory: “Electricity demand exceeds safety load thresholds, indicating a severe transformer capacity overload.”

When a feeder receives a Medium or High risk prediction while ambient heat stress crosses safety bounds ($THI > 28.0$), the system generates a thermal advisory: “Elevated environmental heat stress detected (THI : [value]), accelerating transformer thermal degradation risks.”

Control room engineers can forward these advisories to regional communities, giving residents time to run essential appliances early and postpone heavy power usage during peak hours.

A. Practical Operational Impact

A 2-hour forecast window gives dispatchers critical lead time. Control room staff can adjust feeder configurations, dispatch repair teams, and notify industrial users before circuit breakers trip. Moving from reactive load shedding to predictive load balancing minimizes unserved energy losses and stabilizes distribution circuits.

VIII. CONCLUSION AND FUTURE WORK

A. Conclusion

SmartGrid Sentinel gives regional grid operators in Bangladesh a 2-hour heads-up for load shedding risks. Keeping time sequences unbroken and fitting scaling parameters strictly on past data kept our evaluation leak-free. Evaluating 12 models on 65,983 feeder logs across 141 physical Upazilas showed Informer achieved the best overall results (88.44% Weighted F1-score and 79.00% recall on Medium Risk, training stably over 50 epochs). Connecting Informer to a physical rule parser gives dispatchers plain-text warnings right alongside numerical risk scores.

B. Limitations

Our work has two practical limits. First, telemetry logs cover 141 physical Upazilas across 15 districts in Sylhet and Chattogram rather than a nationwide grid network. Second, our pipeline focuses on a single 2-hour forecast step. Future tests will evaluate longer multi-step prediction horizons over larger geographic regions.

C. Threats to Validity

Primary validity risks involve local weather fluctuations and simplified feeder load assumptions. Restricting test evaluation to chronological splits keeps benchmark numbers realistic for actual control room operation.

D. Future Work

We plan to use Graph Neural Networks (GNNs) to map physical substation layouts and feeder line connections. Topology-aware GNNs will help dispatchers spot cascading blackout risks across linked transmission networks.

REFERENCES

- [1] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, pp. 11106–11115.
- [2] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Advances in Neural Information Processing Systems*, 2021, pp. 22419–22430.
- [3] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *International Conference on Learning Representations*, 2023.
- [4] J. Wu, H. Wu, and M. Long, “TimesNet: Temporal 2D-variation modeling for general time series analysis,” in *International Conference on Learning Representations*, 2023.
- [5] B. Lim, S. O. Arik, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.
- [8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [10] S. Xingjian, Z. Chen, H. Wang, D. Y. Yeung, W. K. Wong, and W. c. Woo, “Convolutional LSTM network: A machine learning approach for precipitation nowcasting,” in *Advances in Neural Information Processing Systems*, 2015, pp. 802–810.
- [11] M. R. I. Sheikh, M. S. Islam, and M. S. Rahman, “Power sector of Bangladesh: Present status and future challenges,” in *International Conference on Electrical, Computer and Communication Engineering*, 2019, pp. 1–6.
- [12] A. S. M. M. H. Rashid, M. A. A. Mahmud, and M. M. Rahman, “An automated load shedding management system using smart meters in smart grid,” in *International Conference on Informatics, Electronics & Vision*, 2016, pp. 53–58.
- [13] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [14] R. J. Hyndman and G. Athanasopoulos, *Forecasting: principles and practice*. OTexts, 2018.
- [15] R. S. Michalski, “A theory and methodology of inductive learning,” *Machine Learning: An Artificial Intelligence Approach*, vol. 1, pp. 83–134, 1983.
- [16] Open-Meteo, “Open-meteo weather api,” <https://open-meteo.com>, 2024.
- [17] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” in *NIPS Deep Learning Workshop*, 2014.
- [18] M. S. H. Lipu, M. A. Hannan, T. F. Karim, and A. Hussain, “Intelligent grid monitoring and load forecasting for Bangladesh power sector using machine learning,” *Energy Reports*, vol. 9, pp. 240–252, 2023.

- [19] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on Explainable Artificial Intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [20] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2018, pp. 95–104.